



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

8

The Game Loop and Real-Time Simulation

Games are real-time, dynamic, interactive computer simulations. As such, *time* plays an incredibly important role in any electronic game. There are many different kinds of time to deal with in a game engine—real time, game time, the local timeline of an animation, the actual CPU cycles spent within a particular function, and the list goes on. Every engine system might define and manipulate time differently. We must have a solid understanding of all the ways time can be used in a game. In this chapter, we'll take a look at how real-time, dynamic simulation software works and explore the common ways in which time plays a role in such a simulation.

8.1 The Rendering Loop

In a graphical user interface (GUI), of the sort found on a Windows PC or a Macintosh, the majority of the screen's contents are static. Only a small part of any one window is actively changing appearance at any given moment. Because of this, graphical user interfaces have traditionally been drawn on-screen via a technique known as *rectangle invalidation*, in which only the small portions of the screen whose contents have actually changed are redrawn. Older 2D video games used similar techniques to minimize the number of pixels that needed to be drawn.

Real-time 3D computer graphics are implemented in an entirely different way. As the camera moves about in a 3D scene, the *entire contents* of the screen or window change continually, so the concept of invalid rectangles no longer applies. Instead, an illusion of motion and interactivity is produced in much the same way that a movie produces it—by presenting the viewer with a series of still images in rapid succession.

Obviously, producing a rapid succession of still images on-screen requires a loop. In a real-time rendering application, this is sometimes known as the *render loop*. At its simplest, a rendering loop is structured as follows:

```
while (!quit)
{
    // Update the camera transform based on interactive
    // inputs or by following a predefined path.
    updateCamera();

    // Update positions, orientations and any other
    // relevant visual state of any dynamic elements
    // in the scene.
    updateSceneElements();

    // Render a still frame into an off-screen frame
    // buffer known as the "back buffer".
    renderScene();

    // Swap the back buffer with the front buffer, making
    // the most recently rendered image visible
    // on-screen. (Or, in windowed mode, copy (blit) the
    // back buffer's contents to the front buffer.
    swapBuffers();
}
```

8.2 The Game Loop

A game is composed of many interacting subsystems, including device I/O, rendering, animation, collision detection and resolution, optional rigid body dynamics simulation, multiplayer networking, audio, and the list goes on. Most game engine subsystems require periodic *servicing* while the game is running. However, the *rate* at which these subsystems need to be serviced varies from subsystem to subsystem. Animation typically needs to be updated at a rate of 30 or 60 Hz, in synchronization with the rendering subsystem. However, a dynamics (physics) simulation may actually require more frequent updates (e.g., 120 Hz). Higher-level systems, like AI, might only need to be

served once or twice per second, and they needn't necessarily be synchronized with the rendering loop at all.

There are a number of ways to implement the periodic updating of our game engine subsystems. We'll explore some of the possible architectures in a moment. But for the time being, let's stick with the simplest way to update our engine's subsystems—using a single loop to update everything. Such a loop is often called the *game loop*, because it is the master loop that services every subsystem in the engine.

8.2.1 A Simple Example: Pong

Pong is a well-known genre of table tennis video games that got its start in 1958, in the form of an analog computer game called *Tennis for Two*, created by William A. Higinbotham at the Brookhaven National Laboratory and displayed on an oscilloscope. The genre is best known by its later incarnations on digital computers—the Magnavox Oddyssey game *Table Tennis* and the Atari arcade game *Pong*.

In pong, a ball bounces back and forth between two movable vertical paddles and two fixed horizontal walls. The human players control the positions of the paddles via control wheels. (Modern re-implementations allow control via a joystick, the keyboard or some other human interface device.) If the ball passes by a paddle without striking it, the other team wins the point and the ball is reset for a new round of play.

The following pseudocode demonstrates what the game loop of a pong game might look like at its core:

```
void main() // Pong
{
    initGame();

    while (true) // game loop
    {
        readHumanInterfaceDevices();

        if (quitButtonPressed())
        {
            break; // exit the game loop
        }

        movePaddles();

        moveBall();

        collideAndBounceBall();
    }
}
```



```

        if (ballImpactedSide(LEFT_PLAYER))
        {
            incremenentScore(RIGHT_PLAYER);
            resetBall();
        }
        else if (ballImpactedSide(RIGHT_PLAYER))
        {
            incrementScore(LEFT_PLAYER);
            resetBall();
        }

        renderPlayfield();
    }
}

```

Clearly this example is somewhat contrived. The original pong games were certainly not implemented by redrawing the entire screen at a rate of 30 frames per second. Back then, CPUs were so slow that they could barely muster the power to draw two lines for the paddles and a box for the ball in real time. Specialized 2D sprite hardware was often used to draw moving objects on-screen. However, we're only interested in the concepts here, not the implementation details of the original Pong.

As you can see, when the game first runs, it calls `initGame()` to do whatever set-up might be required by the graphics system, human I/O devices, audio system, etc. Then the main game loop is entered. The statement `while (true)` tells us that the loop will continue forever, unless interrupted internally. The first thing we do inside the loop is to read the human interface device(s). We check to see whether either human player pressed the “quit” button—if so, we exit the game via a `break` statement. Next, the positions of the paddles are adjusted slightly upward or downward in `movePaddles()`, based on the current deflection of the control wheels, joysticks or other I/O devices. The function `moveBall()` adds the ball's current velocity vector to its position in order to find its new position next frame. In `collideAndBounceBall()`, this position is then checked for collisions against both the fixed horizontal walls and the paddles. If collisions are detected, the ball's position is recalculated to account for any bounce. We also note whether the ball impacted either the left or right edge of the screen. This means that it missed one of the paddles, in which case we increment the other player's score and reset the ball for the next round. Finally, `renderPlayfield()` draws the entire contents of the screen.

8.3 Game Loop Architectural Styles

Game loops can be implemented in a number of different ways—but at their core, they usually boil down to one or more simple loops, with various embellishments. We'll explore a few of the more common architectures below.

8.3.1 Windows Message Pumps

On a Windows platform, games need to service messages from the Windows operating system in addition to servicing the various subsystems in the game engine itself. Windows games therefore contain a chunk of code known as a *message pump*. The basic idea is to service Windows messages whenever they arrive and to service the game engine only when no Windows messages are pending. A message pump typically looks something like this:

```
while (true)
{
    // Service any and all pending Windows messages.
    MSG msg;

    while (PeekMessage(&msg, nullptr, 0, 0) > 0)
    {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    // No more Windows messages to process -- run one
    // iteration of our "real" game loop.
    RunOneIterationOfGameLoop();
}
```

One of the side-effects of implementing the game loop like this is that Windows messages take precedence over rendering and simulating the game. As a result, the game will temporarily freeze whenever you resize or drag the game's window around on the desktop.

8.3.2 Callback-Driven Frameworks

Most game engine subsystems and third-party game middleware packages are structured as *libraries*. A library is a suite of functions and/or classes that can be called in any way the application programmer sees fit. Libraries provide maximum flexibility to the programmer. But, libraries are sometimes difficult to use, because the programmer must understand how to properly use the functions and classes they provide.

In contrast, some game engines and game middleware packages are structured as *frameworks*. A framework is a partially constructed application—the programmer completes the application by providing custom implementations of missing functionality within the framework (or overriding its default behavior). But he or she has little or no control over the overall flow of control within the application, because it is controlled by the framework.

In a framework-based rendering engine or game engine, the main game loop has been written for us, but it is largely empty. The game programmer can write callback functions in order to “fill in” the missing details. The OGRE rendering engine is an example of a library that has been wrapped in a framework. At the lowest level, OGRE provides functions that can be called directly by a game engine programmer. However, OGRE also provides a framework that encapsulates knowledge of how to use the low-level OGRE library effectively. If the programmer chooses to use the OGRE framework, he or she derives a class from `Ogre::FrameListener` and overrides two virtual functions: `frameStarted()` and `frameEnded()`. As you might guess, these functions are called before and after the main 3D scene has been rendered by OGRE, respectively. The OGRE framework’s implementation of its internal game loop looks something like the following pseudocode. (See `Ogre::Root::renderOneFrame()` in *OgreRoot.cpp* for the actual source code.)

```
while (true)
{
    for (each frameListener)
    {
        frameListener.frameStarted();
    }

    renderCurrentScene();

    for (each frameListener)
    {
        frameListener.frameEnded();
    }

    finalizeSceneAndSwapBuffers();
}
```

A particular game’s frame listener implementation might look something like this.

```

class GameFrameListener : public Ogre::FrameListener
{
public:
    virtual void frameStarted(const FrameEvent& event)
    {
        // Do things that must happen before the 3D scene
        // is rendered (i.e., service all game engine
        // subsystems).
        pollJoypad(event);
        updatePlayerControls(event);
        updateDynamicsSimulation(event);
        resolveCollisions(event);
        updateCamera(event);

        // etc.
    }

    virtual void frameEnded(const FrameEvent& event)
    {
        // Do things that must happen after the 3D scene
        // has been rendered.
        drawHud(event);

        // etc.
    }
};

```

8.3.3 Event-Based Updating

In games, an *event* is any interesting change in the state of the game or its environment. Some examples include: the human player pressing a button on the joypad, an explosion going off, an enemy character spotting the player, and the list goes on. Most game engines have an *event system*, which permits various engine subsystems to register interest in particular kinds of events and to respond to those events when they occur (see Section 16.8 for details). A game's event system is usually very similar to the event/messaging system underlying virtually all graphical user interfaces (for example, Microsoft Windows' window messages, the event handling system in Java's AWT or the services provided by C#'s delegate and event keywords).

Some game engines leverage their event system in order to implement the periodic servicing of some or all of their subsystems. For this to work, the event system must permit events to be posted into the future—that is, to be queued for later delivery. A game engine can then implement periodic updating by simply posting an event. In the event handler, the code can perform

whatever periodic servicing is required. It can then post a new event $1/30$ or $1/60$ of a second into the future, thus continuing the periodic servicing for as long as it is required.

8.4 Abstract Timelines

In game programming, it can be extremely useful to think in terms of *abstract timelines*. A timeline is a continuous, one-dimensional axis whose origin ($t = 0$) can lie at any arbitrary location relative to other timelines in the system. A timeline can be implemented via a simple clock variable that stores absolute time values in either integer or floating-point format.

8.4.1 Real Time

We can think of times measured directly via the CPU's high-resolution timer register (see Section 8.5.3) as lying on what we'll call the *real timeline*. The origin of this timeline is defined to coincide with the moment the CPU was last powered on or reset. It measures times in units of CPU cycles (or some multiple thereof), although these time values can be easily converted into units of seconds by multiplying them by the frequency of the high-resolution timer on the current CPU.

8.4.2 Game Time

We needn't limit ourselves to working with the real timeline exclusively. We can define as many other timeline(s) as we need in order to solve the problems at hand. For example, we can define a *game timeline* that is technically independent of real time. Under normal circumstances, game time coincides with real time. If we wish to pause the game, we can simply stop updating the game timeline temporarily. If we want our game to go into slow motion, we can update the game clock more slowly than the real-time clock. All sorts of effects can be achieved by scaling and warping one timeline relative to another.

Pausing or slowing down the game clock is also a highly useful debugging tool. To track down a visual anomaly, a developer can pause game time in order to freeze the action. Meanwhile, the rendering engine and debug fly-through camera can continue to run, as long as they are governed by a different clock (either the *real-time clock* or a separate *camera clock*). This allows the developer to fly the camera around the game world to inspect it from any angle desired. We can even support single-stepping the game clock, by advancing the game clock by one target frame interval (e.g., $1/30$ of a second) each time

a “single-step” button is pressed on the joypad or keyboard while the game is in a paused state.

When using the approach described above, it’s important to realize that the game loop is still running when the game is paused—only the game clock has stopped. Single-stepping the game by adding $1/30$ of a second to a paused game clock is not the same thing as setting a breakpoint in your main loop, and then hitting the F5 key repeatedly to run one iteration of the loop at a time. Both kinds of single-stepping can be useful for tracking down different kinds of problems. We just need to keep the differences between these approaches in mind.

8.4.3 Local and Global Timelines

We can envision all sorts of other timelines. For example, an animation clip or audio clip might have a *local timeline*, with its origin ($t = 0$) defined to coincide with the start of the clip. The local timeline measures how time progressed when the clip was originally authored or recorded. When the clip is played back in-game, we needn’t play it at the original rate. We might want to speed up an animation, or slow down an audio sample. We can even play an animation backwards by running its local clock in reverse.

Any one of these effects can be visualized as a *mapping* between the local timeline and a global timeline, such as real time or game time. To play an animation clip back at its originally authored speed, we simply map the start of the animation’s local timeline ($t = 0$) onto the desired start time ($\tau = \tau_{\text{start}}$) along the global timeline. This is shown in Figure 8.1.

To play an animation clip back at half speed, we can imagine scaling the local timeline to twice its original size prior to mapping it onto the global timeline. To accomplish this, we simply keep track of a time scale factor or playback rate R , in addition to the clip’s global start time τ_{start} . This is illustrated in Figure 8.2. A clip can even be played in reverse, by using a negative time scale ($R < 0$) as shown in Figure 8.3.

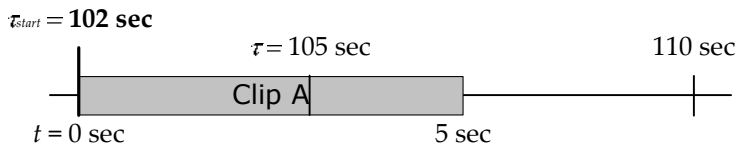


Figure 8.1. Playing an animation clip can be visualized as mapping its local timeline onto the global game timeline.

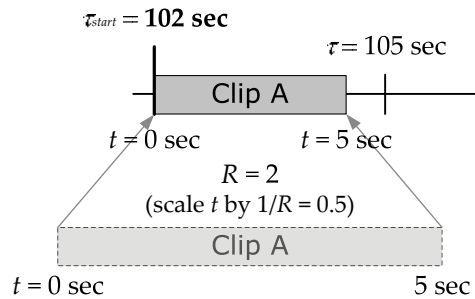


Figure 8.2. Animation playback speed can be controlled by simply scaling the local timeline prior to mapping it onto the global timeline.

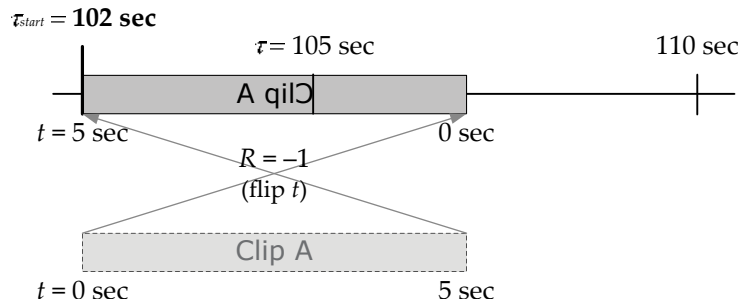


Figure 8.3. Playing an animation in reverse is like mapping the clip to the global timeline with a time scale of $R = -1$.

8.5 Measuring and Dealing with Time

In this section, we'll investigate some of the subtle and not-so-subtle distinctions between different kinds of timelines and clocks and see how they are implemented in real game engines.

8.5.1 Frame Rate and Time Deltas

The *frame rate* of a real-time game describes how rapidly the sequence of still 3D frames is presented to the viewer. The unit of *Hertz* (Hz), defined as the number of cycles per second, can be used to describe the rate of any periodic process. In games and film, frame rate is typically measured in *frames per second* (FPS), which is the same thing as Hertz for all intents and purposes. Films traditionally run at 24 FPS. Games in North America and Japan are typically rendered at 30 or 60 FPS, because this is the natural refresh rate of the NTSC color television standard used in these regions. In Europe and most of the rest

of the world, games update at 50 FPS, because this is the natural refresh rate of a PAL or SECAM color television signal.

The amount of time that elapses between frames is known as the *frame time*, *time delta* or *delta time*. This last term is commonplace because the duration between frames is often represented mathematically by the symbol Δt . (Technically speaking, Δt should really be called the *frame period*, since it is the inverse of the *frame frequency*: $T = 1/f$. But, game programmers hardly ever use the term “period” in this context.) If a game is being rendered at exactly 30 FPS, then its delta time is $1/30$ of a second, or 33.3 ms (milliseconds). At 60 FPS, the delta time is half as big, $1/60$ of a second or 16.6 ms. To really know how much time has elapsed during one iteration of the game loop, we need to measure it. We’ll see how this is done below.

We should note here that milliseconds are a common unit of time measurement in games. For example, we might say that the animation system is taking 4 ms to run, which implies that it occupies about 12% of the entire frame ($4/33.3 \approx 0.12$). Other common units include seconds and machine cycles. We’ll discuss time units and clock variables in more depth below.

8.5.2 From Frame Rate to Speed

Let’s imagine that we want to make a spaceship fly through our game world at a constant speed of 40 meters per second (or in a 2D game, we might specify this as 40 *pixels* per second). One simple way to accomplish this is to multiply the ship’s speed v (measured in meters per second) by the duration of one frame Δt (measured in seconds), yielding a change in position $\Delta x = v\Delta t$ (which is measured in *meters per frame*). This position delta can then be added to the ship’s current position x_1 , in order to find its position next frame: $x_2 = x_1 + \Delta x = x_1 + v\Delta t$.

This is actually a simple form of *numerical integration* known as the *explicit Euler* method (see Section 13.4.4). It works well as long as the speeds of our objects are roughly constant. To handle variable speeds, we need to resort to somewhat more complex integration methods. But all numerical integration techniques make use of the elapsed frame time Δt in one way or another. So it is safe to say that the *perceived speeds* of the objects in a game are dependent upon the frame duration, Δt . Hence a central problem in game programming is to determine a suitable value for Δt . In the sections that follow, we’ll discuss various ways of doing this.

8.5.2.1 Old-School CPU-Dependent Games

In many early video games, no attempt was made to measure how much real time had elapsed during the game loop. The programmers would essentially

ignore Δt altogether and instead specify the speeds of objects directly in terms of meters (or pixels, or some other distance unit) per *frame*. In other words, they were, perhaps unwittingly, specifying object speeds in terms of $\Delta x = v\Delta t$, instead of in terms of v .

The net effect of this simplistic approach was that the perceived speeds of the objects in these games were entirely dependent upon the frame rate that the game was actually achieving on a particular piece of hardware. If this kind of game were to be run on a computer with a faster CPU than the machine for which it was originally written, the game would appear to be running in fast forward. For this reason, I'll call these games *CPU-dependent games*.

Some older PCs provided a "Turbo" button to support these kinds of games. When the Turbo button was pressed, the PC would run at its fastest speed, but CPU-dependent games would run in fast forward. When the Turbo button was not pressed, the PC would mimic the processor speed of an older generation of PCs, allowing CPU-dependent games written for those PCs to run properly.

8.5.2.2 Updating Based on Elapsed Time

To make our games CPU-independent, we must measure Δt in some way, rather than simply ignoring it. Doing this is quite straightforward. We simply read the value of the CPU's high-resolution timer twice—once at the beginning of the frame and once at the end. Then we subtract, producing an accurate measure of Δt for the frame that has just passed. This delta is then made available to all engine subsystems that need it, either by passing it to every function that we call from within the game loop or by storing it in a global variable or encapsulating it within a singleton class of some kind. (We'll describe the CPU's high-resolution timer in more detail in Section 8.5.3.)

The approach outlined above is used by many game engines. In fact, I am tempted to go out on a limb and say that *most* game engines use it. However, there is one big problem with this technique: We are using the measured value of Δt taken during frame k as an estimate of the duration of the *upcoming* frame ($k + 1$). This isn't necessarily very accurate. (As they say in investing, "past performance is not a guarantee of future results.") Something might happen next frame that causes it to take much more time (or much less) than the current frame. We call such an event a *frame-rate spike*.

Using last frame's delta as an estimate of the upcoming frame can have some very real detrimental effects. For example, if we're not careful it can put the game into a "viscious cycle" of poor frame times. Let's assume that our physics simulation is most stable when updated once every 33.3 ms (i.e., at 30 Hz). If we get one bad frame, taking say 57 ms, then we might make the

mistake of stepping the physics system *twice* on the next frame, presumably to “cover” the 57 ms that has passed. Those two steps take roughly twice as long to complete as a regular step, causing the *next* frame to be at least as bad as this one was, and possibly worse. This only serves to exacerbate and prolong the problem.

8.5.2.3 Using a Running Average

It is true that game loops tend to have at least some frame-to-frame coherency. If the camera is pointed down a hallway containing lots of expensive-to-draw objects on one frame, there’s a good chance it will still be pointed down that hallway on the next. Therefore, one reasonable approach is to average the frame-time measurements over a small number of frames and use that as the next frame’s estimate of Δt . This allows the game to adapt to a varying frame rate, while softening the effects of momentary performance spikes. The longer the averaging interval, the less responsive the game will be to a varying frame rate, but spikes will have less of an impact as well.

8.5.2.4 Governing the Frame Rate

We can avoid the inaccuracy of using last frame’s Δt as an estimate of this frame’s duration altogether, by flipping the problem on its head. Rather than trying to *guess* at what next frame’s duration will be, we can instead attempt to *guarantee* that every frame’s duration will be exactly 33.3 ms (or 16.6 ms if we’re running at 60 FPS). To do this, we measure the duration of the current frame as before. If the measured duration is less than the ideal frame time, we simply put the main thread to sleep until the target frame time has elapsed. If the measured duration is more than the ideal frame time, we must “take our lumps” and wait for one more whole frame time to elapse. This is called *frame-rate governing*.

Clearly this approach only works when your game’s frame rate is reasonably close to your target frame rate on average. If your game is ping-ponging between 30 FPS and 15 FPS due to frequent “slow” frames, then the game’s quality can degrade significantly. As such, it’s still a good idea to design all engine systems so that they are capable of dealing with arbitrary frame durations. During development, you can leave the engine in “variable frame rate” mode, and everything will work as expected. Later on, when the game is getting closer to achieving its target frame rate consistently, we can switch on frame-rate governing and start to reap its benefits.

Keeping the frame rate consistent can be important for a number of reasons. Some engine systems, such as the numerical integrators used in a physics simulation, operate best when updated at a constant rate. A consistent frame rate

also looks better, and as we'll see in the next section, it can be used to avoid the *tearing* that can occur when the video buffer is updated at a rate that doesn't match the refresh rate of the monitor (see Section 8.5.2.5).

In addition, when elapsed frame times are consistent, features like *record and playback* become a lot more reliable. As its name implies, the record-and-playback feature allows a player's gameplay experience to be recorded and later played back in exactly the same way. This can be a fun game feature, and it's also a valuable testing and debugging tool. For example, difficult-to-find bugs can be reproduced by simply playing back a recorded game that demonstrates the bug.

To implement record and playback, we make note of every relevant event that occurs during gameplay, saving each one in a list along with an accurate time stamp. The list of events can then be replayed with exactly the same timing, using the same initial conditions and an identical initial random seed. In theory, doing this should produce a gameplay experience that is indistinguishable from the original playthrough. However, if the frame rate isn't consistent, things may not happen in exactly the same order. This can cause "drift," and pretty soon your AI characters are flanking when they should have fallen back.

8.5.2.5 Screen Tearing and V-Sync

A visual anomaly known as *screen tearing* occurs when the back buffer is swapped with the front buffer while the screen has only been partially "drawn" by the video hardware. When tearing occurs, a portion of the screen shows the new image, while the remainder shows the old one. To avoid tearing, many rendering engines wait for the *vertical blanking interval* of the monitor before swapping buffers.

Older CRT monitors and TVs "draw" the contents of the in-memory frame buffer by exciting phosphors on the screen via a beam of electrons that scans from left-to-right and top-to-bottom. On such displays, the v-blank interval is the time during which the electron gun is "blanked" (turned off) while it is being reset to the top-left corner of the screen. Modern LCD, plasma and LED displays no longer use an electron beam, and they don't require any time between finishing the draw of the last scan line of one frame and the first scan line of the next. But the v-blank interval still exists, in part because video standards were established when CRTs were the norm, and in part because of the need to support older displays.

Waiting for the v-blank interval is called *v-sync*. It is really just another form of frame-rate governing, because it effectively clamps the frame rate of the main game loop to a multiple of the screen's refresh rate. For example, on an NTSC monitor that refreshes at a rate of 60 Hz, the game's real update rate

is effectively quantized to a multiple of $1/60$ of a second. If more than $1/60$ of a second elapses between frames, we must wait until the next v-blank interval, which means waiting $2/60$ of a second (30 FPS). If we miss two v-blanks, then we must wait a total of $3/60$ of a second (20 FPS) and so on. Also, be careful not to make assumptions about the frame rate of your game, even when it is synchronized to the v-blank interval; if your game supports them, you must keep in mind that the PAL and SECAM standards are based around an update rate of 50 Hz, not 60 Hz.

8.5.3 Measuring Real Time with a High-Resolution Timer

We've talked a lot about measuring the amount of real "wall clock" time that elapses during each frame. In this section, we'll investigate how such timing measurements are made in detail.

Most operating systems provide a function for querying the system time, such as the C standard library function `time()`. However, such functions are not suitable for measuring elapsed times in a real-time game because they do not provide sufficient resolution. For example, `time()` returns an integer representing the number of *seconds* since midnight, January 1, 1970, so its resolution is one second—far too coarse, considering that a frame takes only tens of milliseconds to execute.

All modern CPUs contain a *high-resolution timer*, which is usually implemented as a hardware register that counts the number of CPU cycles (or some multiple thereof) that have elapsed since the last time the processor was powered on or reset. This is the timer that we should use when measuring elapsed time in a game, because its resolution is usually on the order of the duration of a few CPU cycles. For example, on a 3 GHz Pentium processor, the high-resolution timer increments once per CPU cycle, or 3 billion times per second. Hence the resolution of the high-res timer is $1/3 \text{ billion} = 3.33 \times 10^{-10} \text{ seconds} = 0.333 \text{ ns}$ (one-third of a nanosecond). This is more than enough resolution for all of our time-measurement needs in a game.

Different microprocessors and different operating systems provide different ways to query the high-resolution timer. On a Pentium, a special instruction called `rdtsc` (read time-stamp counter) can be used, although the Win32 API wraps this facility in a pair of functions: `QueryPerformanceCounter()` reads the 64-bit counter register and `QueryPerformanceFrequency()` returns the number of counter increments per second for the current CPU. On a PowerPC architecture, such as the chips found in the Xbox 360 and PlayStation 3, the instruction `mftb` (move from time base register) can be used to read the two 32-bit time base registers, while on other PowerPC architectures, the instruction `mfspr` (move from special-purpose register) is used instead.

A CPU's high-resolution timer register is 64 bits wide on most processors, to ensure that it won't wrap too often. The largest possible value of a 64-bit unsigned integer is $0xFFFFFFFFFFFFFFFF \approx 1.8 \times 10^{19}$ clock ticks. So, on a 3 GHz Pentium processor that updates its high-res timer once per CPU cycle, the register's value will wrap back to zero once every 195 years or so—definitely not a situation we need to lose too much sleep over. In contrast, a 32-bit integer clock will wrap after only about 1.4 seconds at 3 GHz.

8.5.3.1 High-Resolution Clock Drift

Be aware that even timing measurements taken via a high-resolution timer can be inaccurate in certain circumstances. For example, on some multi-core processors, the high-resolution timers are independent on each core, and they can (and do) drift apart. If you try to compare absolute timer readings taken on different cores to one another, you might end up with some strange results—even negative time deltas. Be sure to keep an eye out for these kinds of problems.

8.5.4 Time Units and Clock Variables

Whenever we measure or specify time durations in a game, we have two choices to make:

1. What *time units* should be used? Do we want to store our times in seconds, or milliseconds, or machine cycles...or in some other unit?
2. What *data type* should be used to store time measurements? Should we employ a 64-bit integer, or a 32-bit integer, or a 32-bit floating point variable?

The answers to these questions depend on the intended purpose of a given measurement. This gives rise to two more questions: How much precision do we need? And what range of magnitudes do we expect to be able to represent?

8.5.4.1 64-Bit Integer Clocks

We've already seen that a 64-bit unsigned integer clock, measured in machine cycles, supports both an extremely high precision (a single cycle is 0.333 ns in duration on a 3 GHz CPU) and a broad range of magnitudes (a 64-bit clock wraps once roughly every 195 years at 3 GHz). So this is the most flexible time representation, presuming you can afford 64 bits worth of storage.

8.5.4.2 32-Bit Integer Clocks

When measuring relatively short durations with high precision, we can turn to a 32-bit integer clock, measured in machine cycles. For example, to profile the performance of a block of code, we might do something like this:

```
// Grab a time snapshot.
U64 begin_ticks = readHiResTimer();

// This is the block of code whose performance we wish
// to measure.
doSomething();
doSomethingElse();
nowReallyDoSomething();

// Measure the duration.
U64 end_ticks = readHiResTimer();
U32 dt_ticks = static_cast<U32>(end_ticks - begin_ticks);

// Now use or cache the value of dt_ticks...
```

Notice that we still store the raw time measurements in 64-bit integer variables. Only the time delta `dt` is stored in a 32-bit variable. This circumvents potential problems with wrapping at the 32-bit boundary. For example, if `begin_ticks = 0x12345678FFFFFFB7` and `end_ticks = 0x1234567900000039`, then we would measure a negative time delta if we were to truncate the individual time measurements to 32 bits each prior to subtracting them.

8.5.4.3 32-Bit Floating-Point Clocks

Another common approach is to store relatively small time deltas in floating-point format, measured in units of seconds. To do this, we simply multiply a duration measured in CPU cycles by the CPU's clock frequency, which is in cycles per second. For example:

```
// Start off assuming an ideal frame time (30 FPS).
F32 dt_seconds = 1.0f / 30.0f;

// Prime the pump by reading the current time.
U64 begin_ticks = readHiResTimer();

while (true) // main game loop
{
    runOneIterationOfGameLoop(dt_seconds);

    // Read the current time again, and calculate the delta.
```

```

U64 end_ticks = readHiResTimer();

// Check our units: seconds = ticks / (ticks/second)
dt_seconds = (F32)(end_ticks - begin_ticks)
             / (F32)getHiResTimerFrequency();

// Use end_ticks as the new begin_ticks for next frame.
begin_ticks = end_ticks;
}

```

Notice once again that we must be careful to subtract the two 64-bit time measurements *before* converting them into floating-point format. This ensures that we don't store too large a magnitude into a 32-bit floating-point variable.

8.5.4.4 Limitations of Floating-Point Clocks

Recall that in a 32-bit IEEE float, the 23 bits of the mantissa are dynamically distributed between the whole and fractional parts of the value, by way of the exponent (see Section 3.3.1.4). Small magnitudes require only a few bits, leaving plenty of bits of precision for the fraction. But once the magnitude of our clock grows too large, its whole part eats up more bits, leaving fewer bits for the fraction. Eventually, even the least-significant bits of the whole part become implicit zeros. This means that we must be cautious when storing *long durations* in a floating-point clock variable. If we keep track of the amount of time that has elapsed since the game was started, a floating-point clock will eventually become inaccurate to the point of being unusable.

Floating-point clocks are usually only used to store relatively short time deltas, measuring at most a few minutes and more often just a single frame or less. If an absolute-valued floating-point clock is used in a game, you will need to reset the clock to zero periodically to avoid accumulation of large magnitudes.

8.5.4.5 Other Time Units

Some game engines allow timing values to be specified in a game-defined unit that is fine-grained enough to permit an integer format to be used (as opposed to requiring a floating-point format), precise enough to be useful for a wide range of applications within the engine, and yet large enough that a 32-bit clock won't wrap too often. One common choice is a 1/300 second time unit. This works well because (a) it is fine-grained enough for many purposes, (b) it only wraps once every 165.7 days and (c) it is an even multiple of both NTSC and PAL refresh rates. A 60 FPS frame would be 5 such units in duration, while a 50 FPS frame would be 6 units in duration.

Obviously a 1/300 second time unit is not precise enough to handle subtle effects, like time-scaling an animation. (If we tried to slow a 30 FPS animation down to less than 1/10 of its regular speed, we'd be out of precision!) So for many purposes, it's still best to use floating-point time units or machine cycles. But a 1/300 second time unit can be used effectively for things like specifying how much time should elapse between the shots of an automatic weapon, or how long an AI-controlled character should wait before starting his patrol, or the amount of time the player can survive when standing in a pool of acid.

8.5.5 Dealing with Breakpoints

When your game hits a breakpoint, its loop stops running and the debugger takes over. However, if your game is running on the same computer that your debugger is running on, then the CPU continues to run, and the real-time clock continues to accrue cycles. A large amount of wall clock time can pass while you are inspecting your code at a breakpoint. When you allow the program to continue, this can lead to a measured frame time many seconds, or even minutes or hours in duration!

Clearly if we allow such a huge delta time to be passed to the subsystems in our engine, bad things will happen. If we are lucky, the game might continue to function properly after lurching forward many seconds in a single frame. Worse, the game might just crash.

A simple approach can be used to get around this problem. In the main game loop, if we ever measure a frame time in excess of some predefined upper limit (e.g., 1 second), we can assume that we have just resumed execution after a breakpoint, and we set the delta-time artificially to 1/30 or 1/60 of a second (or whatever the target frame rate is). In effect, the game becomes frame-locked for one frame, in order to avoid a massive spike in the measured frame duration.

```
// Start off assuming the ideal dt (30 FPS).
F32 dt = 1.0f / 30.0f;

// Prime the pump by reading the current time.
U64 begin_ticks = readHiResTimer();

while (true) // main game loop
{
    updateSubsystemA(dt);
    updateSubsystemB(dt);
    // ...
    renderScene();
    swapBuffers();
}
```